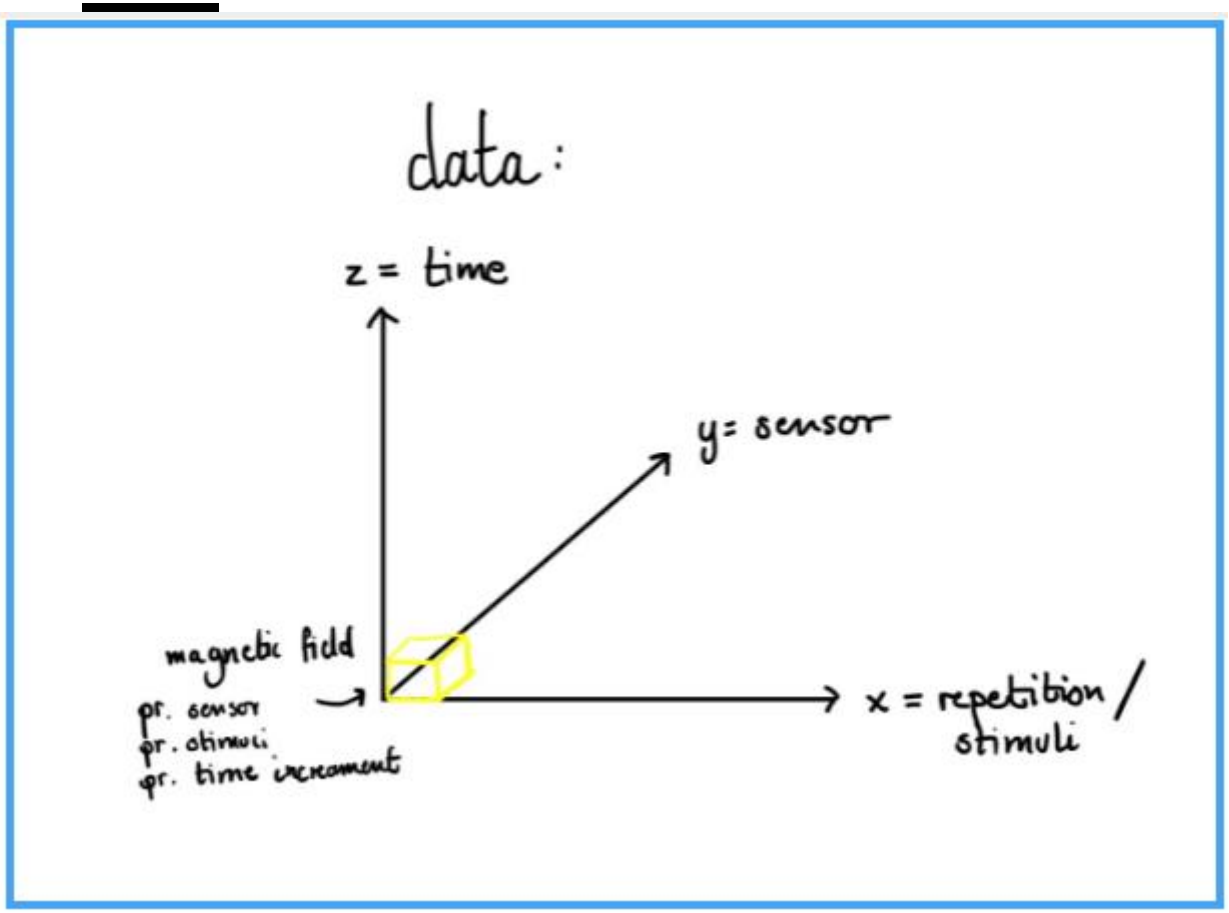
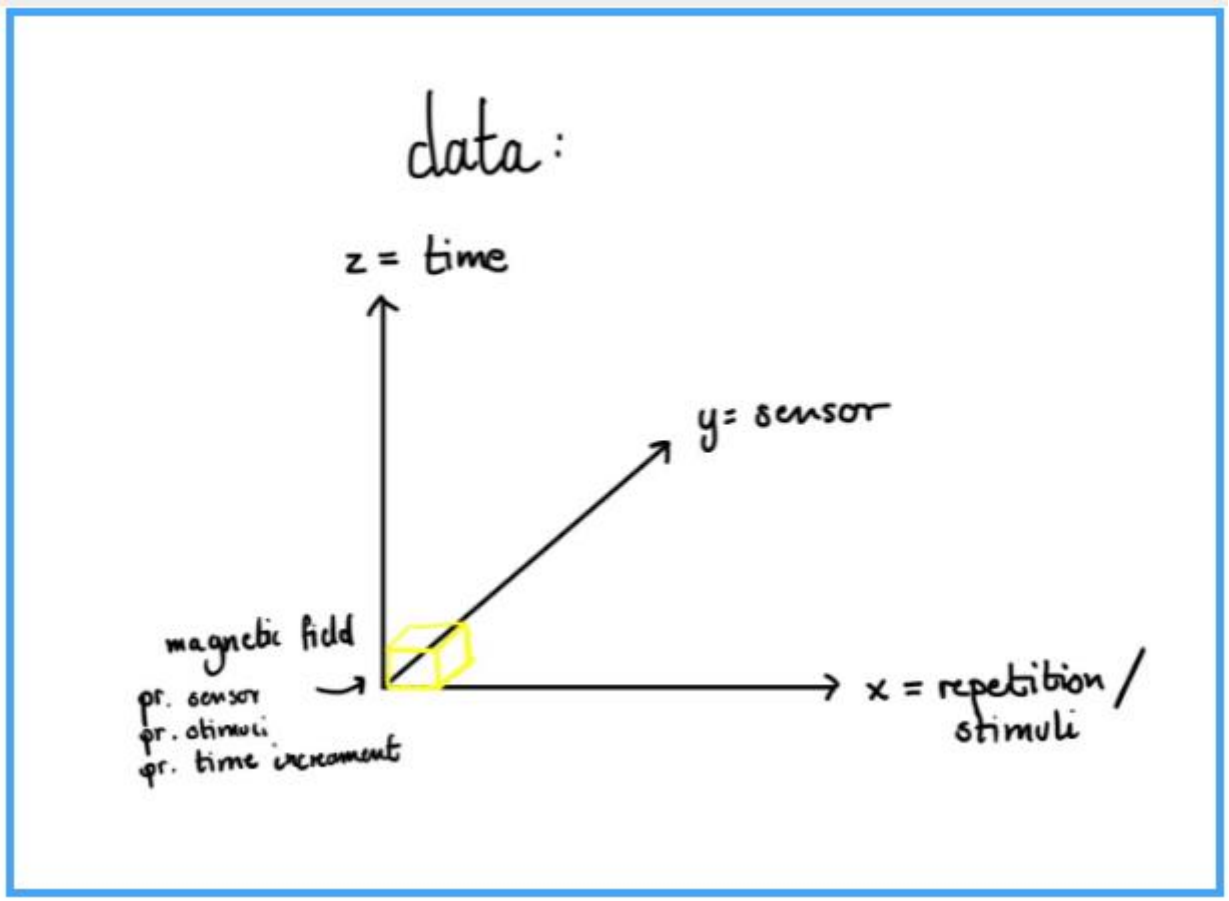
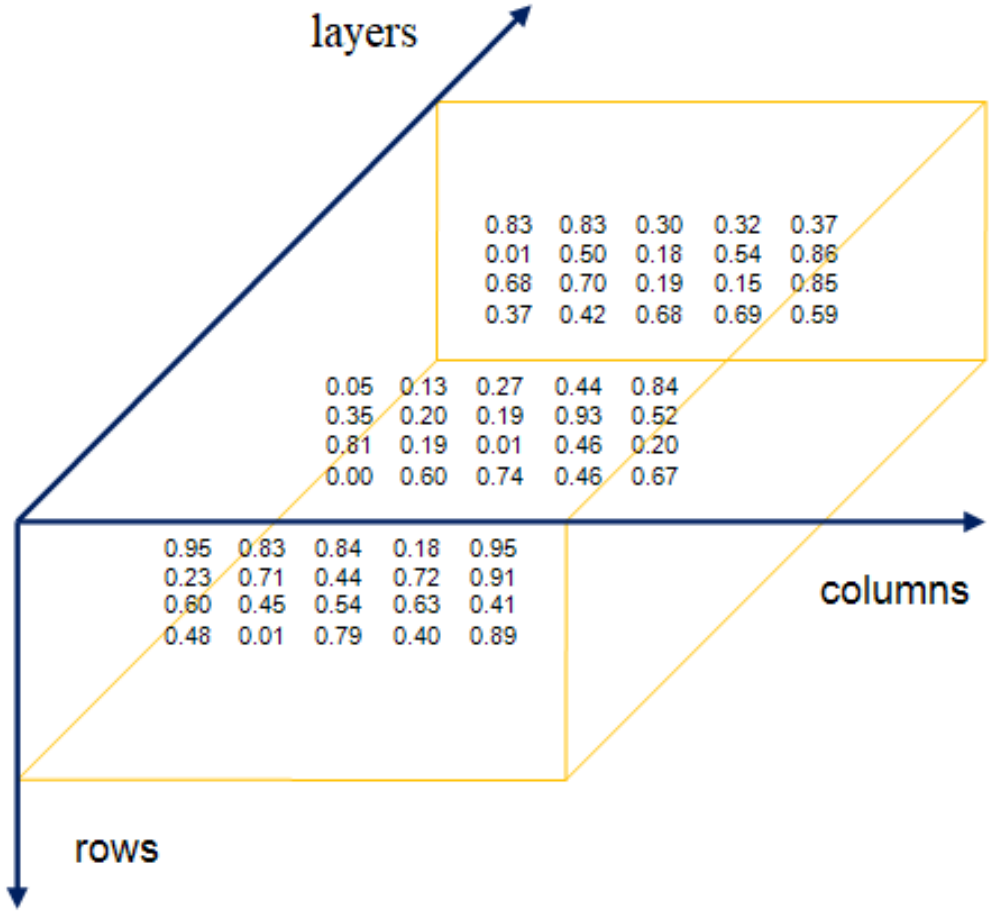
ARRAYS AND MEANS

ARRAYS


data:
z = time
y= sensor
magnetic field
pr. sensor
pr. stimuli
pr. time increament
x = repetition / stimuli

ARRAYS





A 3-D array, with size **4×5×3**, may be described as a "block array" containing three **4×5** matrices (one per page), or also four **5×3** matrices

ARRAYS

- How do they behave?

ARRAYS

```
>>> two_d = np.array([[1,2,3,4],[2,3,4,5]])
>>> two_d
array([[1, 2, 3, 4],
       [2, 3, 4, 5]])
```

ARRAYS

```
>>> two_d = np.array([[1,2,3,4],[2,3,4,5]])
>>> two_d
array([[1, 2, 3, 4],
       [2, 3, 4, 5]])
>>> two_d.shape
(2, 4)
```

Rows, Columns

=

Roman Catholics

ARRAYS

```
>>> two_d = np.array([[1,2,3,4],[2,3,4,5]])
>>> two_d
array([[1, 2, 3, 4],
       [2, 3, 4, 5]])
>>> two_d.shape
(2, 4)
>>> np.mean(two_d, axis=0)
```

ARRAYS

```
>>> two_d = np.array([[1,2,3,4],[2,3,4,5]])
>>> two_d
array([[1, 2, 3, 4],
       [2, 3, 4, 5]])
>>> two_d.shape
(2, 4)
>>> np.mean(two_d, axis=0)
array([1.5, 2.5, 3.5, 4.5])
```

~~Mean of each column?~~

Takes mean so that axis 0 is collapsed

ARRAYS

```
>>> three_d = np.array([[[1,2,3,4],[2,3,4,5]], [[10,9,8,7], [7,6,5,4]], [[7,6,5,4], [7,8,9,10]]])
>>> three_d
array([[[ 1,  2,  3,  4],
        [ 2,  3,  4,  5]],

       [[10,  9,  8,  7],
        [ 7,  6,  5,  4]],

       [[ 7,  6,  5,  4],
        [ 7,  8,  9, 10]]])
>>> three_d.shape
```

ARRAYS

```
>>> three_d = np.array([[[1,2,3,4],[2,3,4,5]], [[10,9,8,7], [7,6,5,4]], [[7,6,5,4], [7,8,9,10]]])
>>> three_d
array([[[ 1,  2,  3,  4],
        [ 2,  3,  4,  5]],

       [[10,  9,  8,  7],
        [ 7,  6,  5,  4]],

       [[ 7,  6,  5,  4],
        [ 7,  8,  9, 10]]])
>>> three_d.shape
(3, 2, 4)
```

Unexpected?

ARRAYS

```
>>> three_d = np.array([[[1,2,3,4],[2,3,4,5]], [[10,9,8,7], [7,6,5,4]], [[7,6,5,4], [7,8,9,10]]])
>>> three_d
array([[[ 1,  2,  3,  4],
        [ 2,  3,  4,  5]],

       [[10,  9,  8,  7],
        [ 7,  6,  5,  4]],

       [[ 7,  6,  5,  4],
        [ 7,  8,  9, 10]]])
>>> three_d.shape
(3, 2, 4)
>>> np.mean(three_d, axis=0)
```

Takes mean so that axis 0 is collapsed

ARRAYS

```
>>> three_d = np.array([[[1,2,3,4],[2,3,4,5]], [[10,9,8,7], [7,6,5,4]], [[7,6,5,4], [7,8,9,10]]])
>>> three_d
array([[[ 1,  2,  3,  4],
        [ 2,  3,  4,  5]],

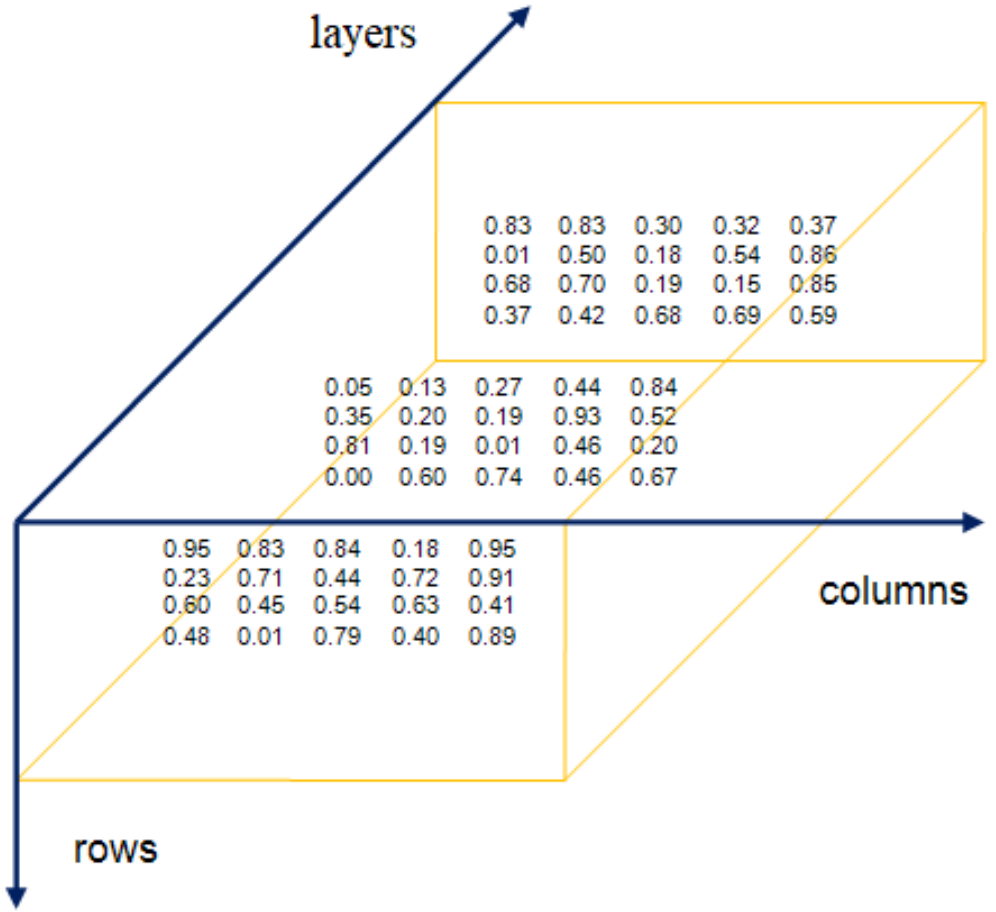
       [[10,  9,  8,  7],
        [ 7,  6,  5,  4]],

       [[ 7,  6,  5,  4],
        [ 7,  8,  9, 10]]])
>>> three_d.shape
(3, 2, 4)
>>> np.mean(three_d, axis=0)
array([[6.        , 5.66666667, 5.33333333, 5.        ],
       [5.33333333, 5.66666667, 6.        , 6.33333333]])
>>> np.mean(three_d, axis=0).shape
(2, 4)
```

ARRAYS

```
>>> np.mean(three_d, axis=0)
```

- What will happen here?
- What are the new dimensions?



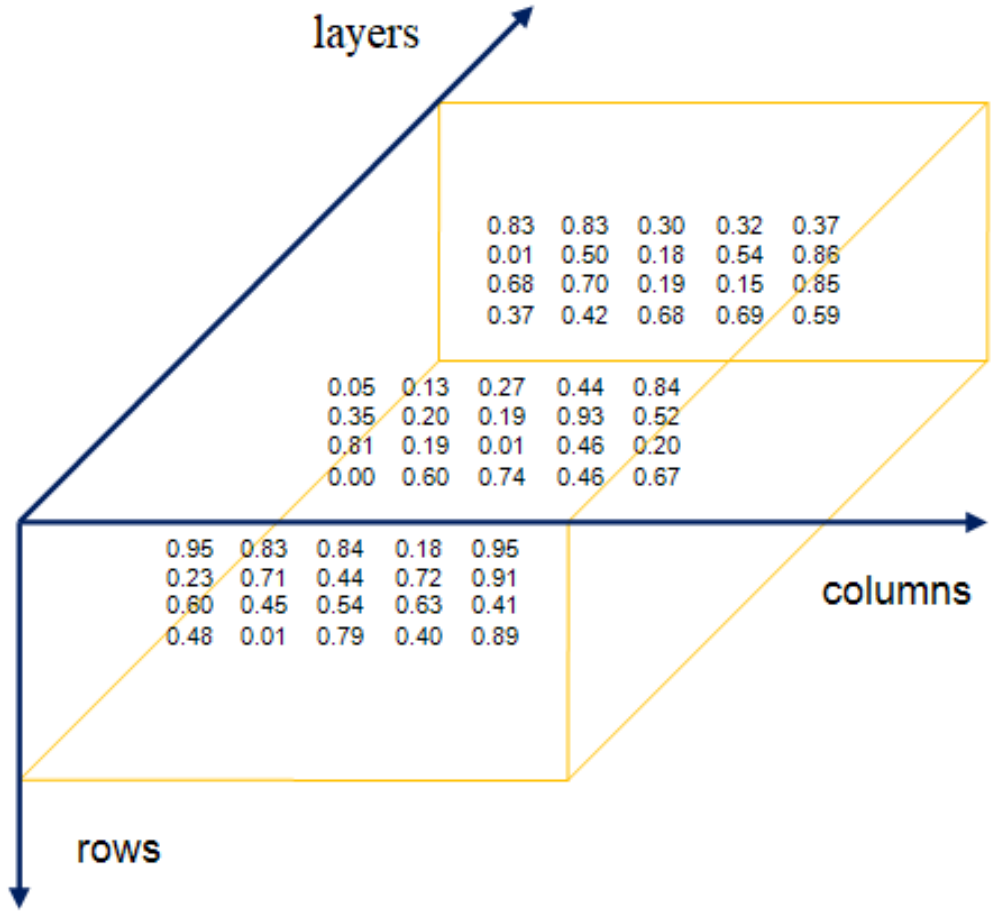
A 3-D array, with size **4×5×3**, may be described as a "block array" containing three **4×5** matrices (one per page), or also four **5×3** matrices

ARRAYS

```
>>> np.mean(three_d, axis=0)
```

- What will happen here?
- What are the new dimensions?
  - *We won't have the dimension[0]*

    -> we won't have depth/layers
  - *New shape = (4,5)*



A 3-D array, with size **4×5×3**, may be described as a "block array" containing three **4×5** matrices (one per page), or also four **5×3** matrices